

U

O

W

CSIT314

Software Development

Methodologies

Subject Admin Introduction



UNIVERSITY
OF WOLLONGONG
AUSTRALIA

Subject Content Introduction



UNIVERSITY
OF WOLLONGONG
AUSTRALIA

Contact details

- Dr. Yudi Zhang (yudi@uow.edu.au)
- Tutor: Mr. Terence Chew (tchew@uow.edu.au)
Chun Huei Liaw (chliaw@uow.edu.au)



Subject objectives (CSIT314)

- On successful completion of this subject, students will be able to, to varying degrees:
 - demonstrate an in-depth understanding of the stages involved in software development and the issues to be considered at each stage.
 - compare and contrast different software development methodologies and process models, and assess their suitability in different development contexts.
 - deploy appropriate theory, practices, and tools for the specification, design, implementation and evaluation of computer-based systems.
 - function effectively as part of a team to apply state-of-the-art software development methodologies to the development of a software system.
 - apply different strategies for assessing and improving software development processes.
 - apply professional standards in software development



Topics

- Introduction and Software Development Lifecycle
- Overview of software process models
- Advanced Unified Modelling Language
- Test driven software development
- Principles and practices of continuous integration and delivery
- DevOps software development practices
- Unified software development process
- Extreme programming
- Kanban software development method
- Data-driven software development
- *And a few others*



Resources

- Lectures
 - PDF files with slides from lectures
- Labs and the project
- Supplementary materials
- One-stop shop: Moodle



Textbook and references

- David Avison, Guy Fitzgerald, Information Systems Development: Methodologies, Techniques and Tools (4th Edition), McGraw-Hill Education, 2006.
- Roger S. Pressman, Software Engineering: A Practitioner's Approach (8th Edition), McGraw-Hill Education, 2014.
- Paul VII, Randal Schaffer, Scrum: A Cleverly Concise and Agile Guide, Pashun Consulting Ltd., 2016
- Craig Larman, Applying UML and Patterns: An Introduction to Object-Oriented Analysis and Design and Iterative Development (3rd Edition), Prentice Hall, 2004
- Philippe Kruchten, The rational unified process: an introduction. Addison-Wesley Professional, 2000
- Mary Beth Chrissis, Michael D. Konrad, Sandra Shrum. CMMI: Guidelines for Process Integration and Product Improvement (2nd Edition), Addison-Wesley Professional, 2006
- Paul Ammann and Jeff Offutt, Introduction to Software Testing (2nd Edition), Cambridge University Press, 2016
- Ian Sommerville, Software Engineering (9th Edition), Cambridge University Press, 2016, <https://cs.gmu.edu/~offutt/softwaretest/>
- IBM DevOps For Dummies®, 3rd IBM Limited Edition, <https://www.ibm.com/downloads/cas/P9NYOK3B>
- Gene Kim, Jez Humble, Patrick Debois, John Willis, The DevOps Handbook: How to Create WorldClass Agility, Reliability, and Security in Technology Organizations, IT Revolution Press, 2016



Assessment

- Lab exercises (10%)
 - Will be assessed in the 4th lab session.
- Group project (40%)
 - Final deliverables
 - Project presentation Q&A - last lab session.
- Final examination
 - Technical Fail
 - To be eligible for a Pass in this subject a student must achieve a mark of at least 40% in the Final Examination.
 - Students who fail to achieve this minimum mark & would have otherwise passed may be given a TF (Technical Fail) for this subject.



Tutorial/Lab

- Each tutorial/lab:
 - First half: an exercise
 - Second half: project
 - Work on the project.
 - Meet “the client” session.
 - Tutor will note your group’s attendance, progress, interactions with “client”, etc. which are the factors considered for the final marking of the project.



Group project

- Up to 6 students per group
- Formation of groups is your responsibility
- **All members MUST be from the same tutorial group.** You should not be forming a project group from various tutorial groups and then requesting to switch tutorial group.
- The group leader needs to submit your group details via Moodle by the end of next week.
- Contact your tutor if you need assistance in forming a group.
- Penalties may be applied if submitting this late.



Academic misconduct

- The Academic Integrity Policy, available at:
<http://www.uow.edu.au/about/policy/UOW058648.html>
- Plagiarism
 - Using another person' s ideas, designs, words or any other work without appropriate acknowledgement ;
 - Re-using one' s own work without appropriate acknowledgement.



Academic misconduct

- copying without appropriate referencing
 - at the very least comment it, but if the code is mostly the work of others you are going to get zero.
- copying from each other
 - Github or sharing codes



Introduction to Software Development and its Lifecycle



UNIVERSITY
OF WOLLONGONG
AUSTRALIA

Software Engineering

- The definition (IEEE standard 610.12-1990)
 - “the application of a systematic, disciplined, quantifiable approach to the development, operation, and maintenance of software; that is, the application of engineering to software”.
 - Tasks: requirements=> design => construction => testing => analysis => maintenance



Software Engineering

- Popularity: more and more major businesses and industries are being run on software and delivered online services.
 - But “With great power there must also come great responsibility” ...
- Quality:
 - Jobs
 - Money
 - Lives
 - ...
- Software Engineering plays a critical part in the software development, in particular with a complex software.



Components of Software Engineering

- PRODUCT
 - The actual software product or system that is built and put into operation
- PROCESS
 - A framework for the tasks that are required to build high-quality software.



What is Engineering

- A body of knowledge used when building things
 - Scheduling
 - Costing
 - Building
 - Testing
 - Communicating
 - Organising
 - ...

It is easy to build something if you have unlimited money and time. A professional differs from an amateur in that they can contain costs and time.



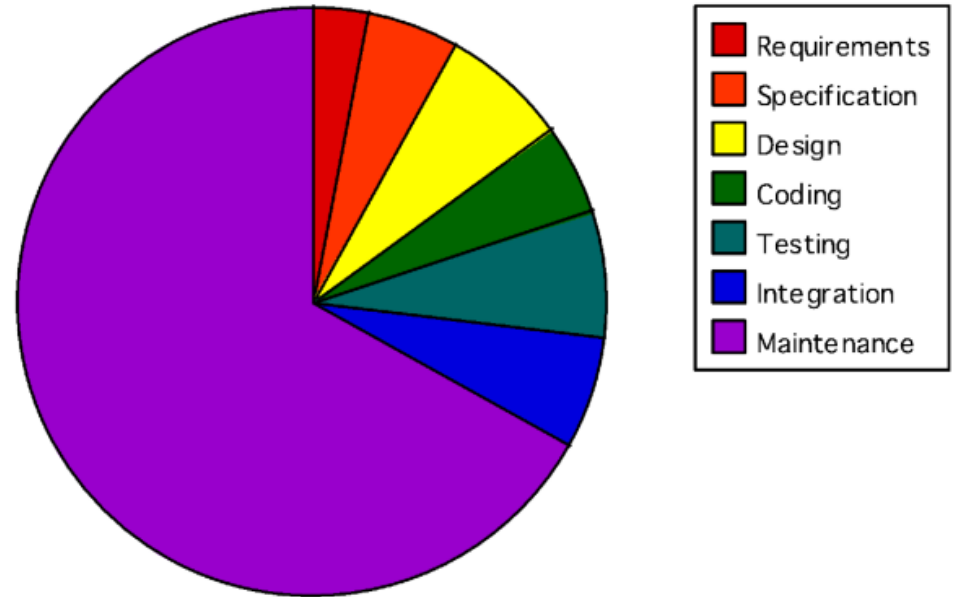
How software is different?

- Software is soft and intangible
- There are no physical laws underlying software behavior
- Software hard to wear out
 - traditional reliability measures don' t apply
- The specification for software continuously changes



Software Development Activities

- Software Engineering plays a critical part in the software development
 - Planning
 - Requirements analysis
 - Design
 - Implementation
 - Verification & Validation
 - Maintenance and evolution



Planning

- Identify business requirement or value
- Analyze feasibility
- Develop work plan
- Staff
- Estimate
- Identify risk



Requirements analysis

- Identify what services are required and the constraints on the system's operation and development.
 - Who are the stakeholders
 - What do they expect
 - Interviews, questionnaires, meeting, etc.
 - Defining the requirements in detail
 - Use cases are the major part of a requirements specification



Design

- Design is concerned with HOW we build a system (not WHAT, more details later)
 - Architectural design
 - Information architecture, data dictionaries, workflow
 - Sub system design
 - Detailed design
 - Persistent data design
 - GUI design



Architecture design

- Concerned with major structure of the software
 - Similar to building architecture (e.g. roof, walls, foundations)
 - Contains sub-systems (air-conditioning unit, water pump)
 - Uses external services (sewerage, water, gas, electric)
 - The most important part of design - if the architecture is wrong, the house will fall down (or software will fall down!)
 - Example: Data-centric, Client-server, Service-oriented



Sub system design

- Describes interfaces/protocols of each subsystem
- Structure of each sub-system
- Structure can describe how each sub-system sub-divides into packages, components



Detailed and Persistent data design

- Detailed design
 - Describes each class, methods, attributes/datatypes
- Persistent data design
 - Describes choice of database, tables, primary and foreign keys



User interface design

- how the GUI will look like
- what tools will be used to build it
- what design (layout) guidelines will be followed
- what style guidelines will be followed



Implementation

- Construction
 - Write the code for the project.
- Installation (or deployment)

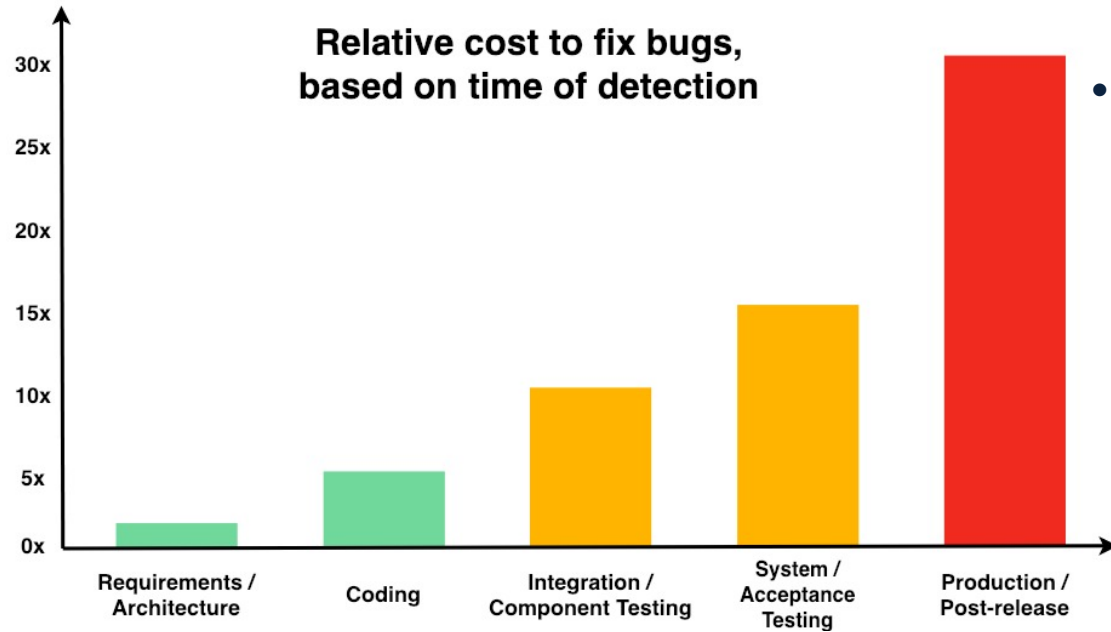


Verification & Validation

- Verification and validation (V & V)
 - The software should conform to its specification (in a correct way)
 - The software should do what the user really requires
- Testing is the most commonly used V & V activity.
- System testing involves executing the system with test cases that are derived from the specification of the real data to be processed by the system.



Why need testing



- Cost of correcting an error in requirement specifications increases as we move through lifecycle phases

Maintenance and evolution

- Changes are inevitable in software development
 - New functionalities requirements emerged at any time in the software development lifecycle
 - Changes in business environments
 - competition, laws, new markets, new customers, etc.
 - Changes in infrastructure environments
 - new servers, new equipment, etc.
 - New soft technology arriving
 - New version of OS, new standards, etc.
 - Bugs need fixing
 - Performance needs improvement



Maintenance and evolution

- Adaptive maintenance
 - Changing the system in response to changes in its environment so it continues to function
- Corrective maintenance
 - Fixing errors & bugs
- Perfective maintenance
 - Changing the system' s functionality to meet changing needs



The eight laws of software evolution

- Law 1: Continuing change
 - system must be continually adapted or it becomes progressively less satisfactory
- Law 2: Increasing complexity
 - system complexity increases unless work is done to maintain or reduce
- Law 3: Self regulation
 - a system to adjust and maintain itself without external intervention, such as feedback loops to adapt to changing conditions, correct errors, and maintain stability over time

The eight laws of software evolution

- Law 4: Conservation of organisational stability
 - organizational stability is preserved despite ongoing changes and improvements to the system
- Law 5: Conservation of familiarity
 - all association (developers, sales personnel and users) must maintain a high level of expertise or proficiency
- Law 6: Continuing growth
 - the functional content must be continually increased



The eight laws of software evolution

- Law 7: Declining quality
 - the quality will appear to be declining unless it is rigorously maintained and adapted to operational environment changes
- Law 8: Feedback system
 - multi-level, multi-loop, multi-agent feedback systems and must be treated as such to achieve significant improvement

